

가볼래 포팅 매뉴얼

클론 후 빌드와 배포, 외부 서비스 등록, 최신 데이터베이스 덤프, 앱 여행 만들기 시연 순서를 정리했다. 2026 년 9 월 28 일 저장소와 운영 서버에서 확인한 값은 확인 범위와 함께 적었고, 새 환경에서 실제 배포하거나 앱 시연을 마친 것으로 표현하지 않았다.

기준 저장소: S15P21E201 / 작성일: 2026 년 9 월 28 일

1. 클론 후 빌드 및 배포

제품 및 버전

구분	저장소에서 확인한 값	근거
운영 서버 OS	Ubuntu 24.04.4 LTS	2026-09-28 운영 서버 /etc/os-release 확인
운영 웹서버	호스트 Nginx 1.24.0 (Ubuntu 패키지). 프론트엔드 이미지 내부는 nginx:alpine 으로 세부 버전을 고정하지 않음	운영 서버 nginx -v, frontend/Dockerfile
운영 컨테이너·배포 도구	Docker 29.7.2, Jenkins 2.568.3 (jenkins/jenkins:lts 이미지), Jenkins JVM Temurin 21.0.12.1	운영 서버 docker --version, Jenkins 컨테이너의 WAR·JVM 버전 확인
백엔드 언어·빌드	Java 17, Gradle Wrapper 9.7.1, Spring Boot 4.0.8	backend/build.gradle, backend/gradle/wrapper/gradle-wrapper.properties
WAS (Java 웹 애플리케이션 실행 서버)	Spring Boot 내장 Tomcat 11.0.25	backend/build.gradle 의 tomcat.version
백엔드 이미지	eclipse-temurin:17-jdk 로 빌드, eclipse-temurin:17-jre 로 실행. 경로 최적화에는 별도 Python 3.13 포함	backend/Dockerfile
운영 백엔드 JVM·DB	실행 중인 백엔드 JVM 은 OpenJDK 17.0.20, PostgreSQL 서버는 16.4	2026-09-28 운영 컨테이너에서 버전 확인
웹 빌드·서빙	node:20-alpine 에서 Expo 웹 export, nginx:alpine 컨테이너에서 정적 파일 제공. Nginx 이미지 세부 버전은 고정되지 않음	frontend/Dockerfile
모바일 앱	Expo ~57.0.22, React Native 0.86.3; 앱 빌드는 EAS	frontend/package.json, frontend/eas.json

구분	저장소에서 확인한 값	근거
	Build(Expo 클라우드 빌드) 설정 사용	
DB-선택 서비스	로컬 Compose 는 PostgreSQL 16-alpine; Kafka 3.8.0 은 events 프로필을 명시할 때만 실행	backend/docker-compose.yml
IDE	버전 지정 없음. Java 17 과 Node 20 을 지원하는 편집기 사용	저장소에 IDE 버전 고정 파일 없음

backend/README.md 에는 Spring Boot 4.0.0 이라는 오래된 표기가 있다. 실제 빌드 선언인 backend/build.gradle 의 4.0.8 을 따른다.

클론과 로컬 빌드

```
git clone https://lab.ssafy.com/s15-bigdata-dist-sub1/S15P21E201.git
cd S15P21E201
git checkout main

cd backend
./gradlew bootJar
docker build -t gabolle-backend:local .

cd ../frontend
npm ci
npm run build:web
docker build -t gabolle-frontend:local \
  --build-arg EXPO_PUBLIC_API_BASE_URL="https://<새-도메인>" \
  --build-arg EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL="https://<새-도메인>" \
  --build-arg EXPO_PUBLIC_GOOGLE_CLIENT_ID="<공개-ID>" \
  --build-arg EXPO_PUBLIC_NAVER_CLIENT_ID="<공개-ID>" \
  --build-arg EXPO_PUBLIC_KAKAO_CLIENT_ID="<공개-ID>" \
  --build-arg EXPO_PUBLIC_APPLE_CLIENT_ID="<공개-ID>" \
  --build-arg EXPO_PUBLIC_KAKAO_MAP_JS_KEY="<공개-지도-키>" .
```

Windows 에서는 backend/gradlew.bat 을 사용한다. 위 docker build 의 <...>는 새 환경에서 발급받은 **공개 식별자**로 바꾼다. npm run build:web 을 먼저 실행할 때도 같은 EXPO_PUBLIC_* 변수를 해당 셀에 설정한다. 이 값들이 빠져도 frontend/Dockerfile 의 검사는 경고만 내고 계속 빌드하므로 생성된 번들의 로그인·지도·API 연결을 확인해야 한다. 서버의 client secret 은 --build-arg 로 전달하지 않는다. 앱은 frontend/eas.json 과 frontend/app.config.js 를 확인해 별도로 빌드한다. 위 명령의 전체 실행 성공 여부는 이 문서 작성 중 검증하지 않았다.

로컬 백엔드와 PostgreSQL 을 함께 띄울 때는 backend/.env.example 을 backend/.env 로 복사하고 실제 비밀값을 입력한 다음 backend/에서 docker compose up -d --build 를 실행한다. POSTGRES_PASSWORD 와 GABOLLE_JWT_SECRET 은 필수다. Compose 는 backend/docker-compose.yml 에 **나열된 변수만** 컨테이너로 전달한다. .env 에 값을 추가하는 것만으로는 전달되지 않는다. 웹 Docker 이미지는 frontend/에서 빌드하며, EXPO_PUBLIC_*는 이미지 빌드 전에 전달해야 한다.

환경 변수와 설정 파일

위치	채워야 할 내용
backend/.env.example, backend/docker-compose.yml	로컬 DB 이름·사용자·비밀번호(POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD), 앱 포트, JWT 서명 비밀값, 메일, 소셜 로그인, 외부 API 키. 실제 .env 는 Git 에 넣지 않는다.
backend/src/main/resources/application.properties, application-dev.properties	서버 프로필, DB/Flyway(기동 시 DB 변경 이력을 적용하는 도구), 인증, 외부 서비스 설정의 변수 읽기와 기본값.
backend/Jenkinsfile	운영 배포 시 Jenkins Credentials(배포 서버의 비밀값 저장소) ID 와 컨테이너 전달값의 실제 대응표. .env.example 보다 항목이 많다.
frontend/.env.example, frontend/Dockerfile, frontend/Jenkinsfile	EXPO_PUBLIC_API_BASE_URL, EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL, Google-Naver-Kakao-Apple 공개 client ID, EXPO_PUBLIC_KAKAO_MAP_JS_KEY, 빌드 식별 정보. EXPO_PUBLIC_*는 웹 빌드 때 번들에 들어가며 컨테이너 실행 때 넣어도 바뀌지 않는다.
frontend/app.json, frontend/app.config.js, frontend/eas.json	앱 식별자와 EAS 빌드 프로필, 앱 빌드 시 필수 URL·OAuth 식별자.
bigData/.env.example, bigData/deploy/.env.example, bigData/config/sources.json	공공데이터·Mapillary 인증, BIMS(부산 버스 위치 데이터) 수집기별 키와 호출 대상.
infra/personalization/.env.example, compose.yaml, Jenkinsfile	PostgreSQL-Redis-MinIO-MLflow-Airflow 인프라의 접속 및 배포 설정.
backend/src/main/resources/db/migration/	Flyway SQL 파일. DB 의 현재 데이터 덤프는 아니다.

DB 와 ERD 확인에 필요한 파일

파일	계정·프로퍼티와 용도
infra/personalization/.env.example	운영 DB 관리자와 app_db, Airflow, MLflow 의 비밀번호 변수명. 값은 비워 둔 견본이다.
infra/personalization/compose.yaml, infra/personalization/postgres/init/01-init-	PostgreSQL 16.4 이미지, DB-계정 초기화와 서비스 간 연결 관계. 운영의 실제 비밀값은 저장소 밖에 둔다.

파일	계정·프로퍼티와 용도
databases.sh	
backend/.env.example, backend/docker- compose.yml	로컬 개발 DB gabolle 과 앱 계정의 변수명, 컨테이너 주입 목록. 운영 app_db 값과 혼동하지 않는다.
backend/src/main/resources/application- dev.properties, backend/Jenkinsfile	GABOLLE_DB_URL, GABOLLE_DB_USERNAME, GABOLLE_DB_PASSWORD, GABOLLE_DB_SCHEMA 를 Spring 데이터소스와 Flyway 에 연결하고 배포 시 주입한다.
backend/src/main/resources/db/migration/, docs/DB-STANDARD.md	현재 PostgreSQL 스키마를 재현·검토할 근거. 테이블 구조는 적용된 Flyway 이력과 실제 DB 를 함께 확인한다.

ref/local-route/LOCAL_ROUTE_ERD_v1.1_FINAL.dbml 은 예전 LOCAL ROUTE 의 **MySQL** ERD 다. 현재 GABOLLE 의 PostgreSQL 운영 스키마로 제출하거나 복원 기준으로 사용하면 안 된다. 현재 스키마를 ERD 로 제출해야 한다면 적용된 Flyway 이력과 운영 DB 의 스키마를 대조해 새로 생성해야 한다.

현재 프로젝트가 공유한 ERD 는 [GABOLLE ERD FigJam](#) 이다. 링크의 보드 제목은 확인했다. ERD 가 실제 운영 스키마와 완전히 일치하는지는 Flyway 이력 및 덤프 복원 결과와 대조해야 한다.

운영 백엔드에서 Jenkins 가 주입하는 값은 GABOLLE_DB_URL, GABOLLE_DB_USERNAME, GABOLLE_DB_PASSWORD, GABOLLE_DB_SCHEMA, GABOLLE_JWT_SECRET, 메일(GABOLLE_MAIL_*), 소셜

로그인(GABOLLE_{GOOGLE,NAVER,KAKAO,APPLE}_*), 사진 저장소(GABOLLE_S3_*), 외부 API

키(GABOLLE_MENU_SCAN_API_KEY, GABOLLE_KMA_SERVICE_KEY, GABOLLE_TRANSIT_SERVICE_KEY,

GABOLLE_EXCHANGE_RATE_AUTH_KEY, GABOLLE_INTERNAL_API_TOKEN) 등이다. 정확한 ID 와 기본값은

backend/Jenkinsfile 의 withCredentials 와 docker run -e 목록을 함께 확인한다. 실제 값은 이 문서에 적지 않는다.

배포 흐름과 특이사항

1. 운영 Jenkins 의 backend/Jenkinsfile 은 **`back/dev`**, frontend/Jenkinsfile 은 **`front/dev`**를 체크아웃한다. main 에 문서를 push 해도 이 두 Job 이 새 코드를 자동 배포하지는 않는다.
2. 백엔드 배포는 실제 DB 에 적용된 Flyway 버전과 새 SQL 의 순서를 배포 전에 검사한다. 이미지 local-route-backend:candidate 를 만들고 컨테이너 상태 URL <http://backend:8080/actuator/health> 확인 후 latest 로 승격한다. 프론트도 local-route-frontend:candidate 를 확인한 뒤 latest 로 승격한다.
3. Jenkins 와 서비스 컨테이너는 local-route-personalization_data_net Docker 네트워크를 공유한다. 호스트 Nginx 는 /api/를 백엔드 8080 으로, /를 프론트엔드 3000 으로 전달한다. /api/의 proxy_pass 끝에 /를 붙이면 경로 접두사가 제거되므로 docs/SERVER-SETUP.md 의 설정을 따른다.

4. Nginx 서버 설정 위치는 `/etc/nginx/sites-available/default`, 업로드 크기 설정 위치는 `/etc/nginx/conf.d/upload-size.conf` 이다. 서버 기록의 설정 경로이며 현재 서버 파일 자체는 확인하지 않았다.
5. DB 주소는 운영 Jenkins 자격증명으로 주입된다. 2026-09-28 실행 중인 백엔드의 연결 대상 DB 명이 `app_db` 임을 실제 컨테이너 환경에서 확인했다. 로컬 Compose 기본 DB 명 `gabolle` 과 다르다. 백업·복원 검증 범위는 [03-db-dump.md](#) 에 기록했다.

운영 서버, Jenkins, EAS 계정에 로그인해 새 환경에서 실제 빌드·배포·상태 확인을 수행한 결과는 이 문서에 포함되지 않는다.

2. 외부 서비스 등록과 활용

아래는 `main` 의 코드와 설정에서 **실제로 참조하는 서비스**다. 발급한 키의 값이나 콘솔 계정은 포함하지 않는다. 제공자 콘솔의 실제 등록 상태, 할당량과 키 유효성은 별도 확인이 필요하다.

서비스	포팅 시 준비할 것	코드/설정 근거
Google, Naver, Kakao 로그인	각 개발자 콘솔에서 앱을 등록하고 웹 콜백 URI 를 허용한다. 공개 client ID 는 <code>EXPO_PUBLIC_{GOOGLE,NAVER,KAKAO}_CLIENT_ID</code> , 서버의 ID 와 secret 은 <code>GABOLLE_{GOOGLE,NAVER,KAKAO}_CLIENT_ID</code> 와 <code>..._CLIENT_SECRET</code> 으로 등록한다. 서버의 <code>GABOLLE_OAUTH_ALLOWED_REDIRECT_URIS</code> 와 정확히 맞춘다.	<code>frontend/.env.example</code> , <code>backend/src/main/resources/application-dev.properties</code> 의 OAuth 설정, <code>docs/GABOLLE_BACKEND_AUTH_HANDOFF.md</code>
Apple 로그인	Apple 개발자 계정에서 Service ID, Team ID, Key ID, .p8 개인키를 준비한다. 서버 변수는 <code>GABOLLE_APPLE_CLIENT_ID</code> , <code>GABOLLE_APPLE_TEAM_ID</code> , <code>GABOLLE_APPLE_KEY_ID</code> , <code>GABOLLE_APPLE_PRIVATE_KEY</code> , 앱 공개 ID 는 <code>EXPO_PUBLIC_APPLE_CLIENT_ID</code> 다. 웹과 iOS 앱의 허용 client ID 범위도 확인한다.	<code>backend/src/main/resources/application-dev.properties</code> 의 Apple 단락, <code>backend/Jenkinsfile</code> , <code>frontend/Dockerfile</code>
Kakao 지도·장소·경로	Kakao Developers 에서 웹 도메인과 앱 사용 환경을 등록한다. 웹·앱 WebView 지도용 JavaScript 키는 <code>EXPO_PUBLIC_KAKAO_MAP_JS_KEY</code> ; 서버의 길찾기·출발지 검색은 <code>GABOLLE_KAKAO_REST_API_KEY</code> 또는 로그인용 Kakao REST ID 를 사용한다. 웹 빌드 환경과 EAS 앱 빌드 환경 각각에 공개 지도 키를 넣는다.	<code>frontend/docs/MAP-RECOVERY.md</code> , <code>frontend/.env.example</code> , <code>backend/src/main/resources/application-dev.properties</code> 의 경로 설정
SSAFY GMS 중계	서비스 이용 권한과 키를 발급받아	<code>backend/src/main/resources/appli</code>

서비스	포팅 시 준비할 것	코드/설정 근거
	GABOLLE_MENU_SCAN_API_KEY 로 배포 자격증명에 등록한다. 메뉴판 읽기와 여행 이름·번역이 같은 키를 참조하며 자연어 도우미도 별도 GABOLLE_ASSISTANT_API_KEY 가 없으면 이 키를 사용한다. 제공되는 모델과 기본 URL 조합을 함께 확인한다.	cation-dev.properties 의 번역·도우미·메뉴판 단락, backend/Jenkinsfile
공공데이터포털	data.go.kr 가입 후 부산 버스 실시간 위치(BIMS), TourAPI 관광정보, 무장애 여행정보, TAGO 정류소·도착정보의 해당 오퍼레이션을 활용신청한다. 일반 인증키와 각 오퍼레이션의 요청주소 를 확인한다. 수집은 DATA_GO_KR_KEY, 서버 TAGO 기능은 GABOLLE_TRANSIT_SERVICE_KEY 를 사용한다. 수집기별 한도와 호출 주기를 확인한다.	bigData/docs/SOURCES.md, bigData/config/sources.json, bigData/.env.example, backend/src/main/resources/applica- tion-dev.properties
기상청 단기에보	기상청 API 허브 에서 별도 인증키를 발급받아 GABOLLE_KMA_SERVICE_KEY 에 등록한다. data.go.kr 키와 호환되지 않는다.	backend/src/main/resources/applica- tion-dev.properties 의 날씨 설정, backend/docker-compose.yml
한국수출입은행 환율	환율 API 인증키를 발급받아 GABOLLE_EXCHANGE_RATE_AUTH_KEY 로 등록한다.	backend/src/main/resources/applica- tion-dev.properties 의 환율 설정
Mapillary 거리 이미지	Mapillary 개발자 페이지 에서 앱을 등록하고 MAPILLARY_TOKEN 을 발급받는다. 수집 전 부산 지역 이미지 범위를 확인한다.	bigData/docs/SOURCES.md, bigData/.env.example
Expo EAS 앱 빌드	Expo 프로젝트 접근 권한과 앱 식별자를 확인하고 frontend/eas.json, frontend/app.json, frontend/app.config.js 의 프로필·필수 URL·공개 ID 를 EAS 환경에 맞춘다.	해당 설정 파일, frontend/Dockerfile 상단 설명
Gmail SMTP	메일 발송을 쓰는 환경에서 발송 계정과 앱 비밀번호, 발신 주소를 GABOLLE_MAIL_*에 등록한다. 기능 비활성 환경은 GABOLLE_MAIL_ENABLED=false 다.	backend/.env.example, backend/Jenkinsfile

사진 저장은 배포 인프라의 **MinIO**(S3 호환 객체 저장소)를 사용한다. 별도 AWS S3 계정을 전제로 만들지 않는다.
접속·버킷·접근 키는 infra/personalization/compose.yaml 과 backend/Jenkinsfile 의 GABOLLE_S3_* 전달을
확인한다.

로그인 제공자에 등록할 주소

현재 웹 도메인을 그대로 사용하는 경우 Google, Naver, Kakao 의 콜백은 각각

`https://j15e201.p.ssafy.io/oauth/google/callback`, `https://j15e201.p.ssafy.io/oauth/naver/callback`,

`https://j15e201.p.ssafy.io/oauth/kakao/callback` 이다. 세 주소는 `backend/.env.example` 의

`GABOLLE_OAUTH_ALLOWED_REDIRECT_URIS` 에 적혀 있다. 새 도메인으로 포팅하면 제공자 콘솔, 프론트의

`EXPO_PUBLIC_OAUTH_CALLBACK_BASE_URL`, 백엔드의 허용 목록을 같은 주소로 바꾼다. Google 은 스킴·대소문자·끝
슬래시까지 일치해야 한다.

Apple 은 웹 도메인, Service ID, Team ID, Key ID, 개인키 외에 콜백 처리 경로를 따로 확인한다. 코드의 기본 허용
목록에는 `/oauth/apple/callback` 과 서버의 `/api/v1/auth/oauth/apple/form-post` 가 모두 있다. 실제 인증 요청에
쓰는 `redirect_uri` 를 실행 환경에서 확인한 다음 Apple 콘솔의 Return URL 과 정확히 일치시킨다. .p8 파일이나
키 문자열은 문서와 Git 에 넣지 않는다.

등록 절차는 [Google OAuth 문서](#), [NAVER 앱 등록 안내](#), [Kakao 로그인 안내](#), [Kakao Maps 안내](#), [Apple 웹 로그인
설정](#), [Expo EAS 환경 변수 안내](#)를 참고한다. 제공자 콘솔의 현재 승인 상태는 확인하지 않았다.

계정과 키를 등록할 때 확인할 순서

1. 새 환경의 도메인, 앱 식별자와 소셜 로그인 콜백 URI 를 먼저 확정한다.
2. 각 제공자 콘솔에 위 값을 등록하고 승인 여부를 확인한다. 발급 키는 팀의 비밀 저장소 또는 Jenkins
Credentials 에 넣는다.
3. 웹 공개 변수는 **Docker 빌드 시점**, 앱 공개 변수는 **EAS 빌드 시점**, 서버 비밀값은 **컨테이너 실행 시점**에
주입한다.
4. 데이터 제공 API 는 승인된 오퍼레이션과 요청주소를 확인한다. 서비스마다 키 형식과 호출 한도가 다르다.
5. 실제 로그인, 지도, 날씨, 데이터 수집, 메일 발송이 새 환경에서 되는지는 기능별로 확인한다. 이 문서는 그
실행 결과를 주장하지 않는다.

`backend/Jenkinsfile` 의 기상청 키 출처 주석은 오래된 내용이다. 현재 서버 설정의 `GABOLLE_KMA_SERVICE_KEY`
설명에 따라 기상청 API 허브에서 발급한다.

3. DB 덤프 최신본

2026-09-28 00:30 UTC(09:30 KST)에 운영 PostgreSQL 의 `app_db` 를 `pg_dump -Fc`(PostgreSQL 사용자 지정 백업
형식)로 새로 추출했다. 실행 중인 백엔드의 DB URL 이 `app_db` 를 가리키는 것을 **DB 명만** 확인했으며 접속
주소·비밀번호는 기록하지 않았다. `pg_dump` 는 일관된 시점의 스냅샷을 읽으며 운영 테이블을 수정하지 않는다.

항목	검증 결과
로컬 원본 파일	<code>exec/app_db_exec_20260928T003005Z.dump</code> (Git 제외)

항목	검증 결과
서버 원본 파일	운영 서버의 사용자 전용 파일로 보관, 권한 umask 077 로 생성
덤프 생성 완료	2026-09-28 00:30:47 UTC
크기	54,406,023 바이트
SHA-256	aa53c0a8c95ce349c51c9f5a550deaf6dd780f2ea43db4d0cfb1914198679032
형식 목록	운영 PostgreSQL 16.4 의 pg_restore -1 성공
전송 무결성	로컬 파일 SHA-256 이 서버 원본과 일치
격리 DB 전체 복원	미실행 . 복원 성공까지 검증한 것으로 표현하지 않는다.

이 덤프는 운영 사용자·인증·위치 등 민감한 데이터를 포함할 수 있다. exec/.gitignore 가 원본 .dump 파일을 Git 에서 제외하며, 메인 브랜치에는 이 설명과 파일의 해시만 올린다. 제출처가 원본을 요구하면 권한자가 별도 보호된 전달 경로를 정해 전송한다. 무단으로 공개 저장소나 메신저에 첨부하지 않는다.

사무국 지급 PEM 으로 운영 DB 덤프 만들기

사무국에서 지급받은 PEM 키가 이 서버에 대한 SSH 접근 권한을 갖는지 먼저 확인한다. 키 파일의 내용은 Git 과 문서에 복사하지 않는다. Windows PowerShell 에서는 지급받은 키 파일을 개인 계정만 읽을 수 있게 보관한 뒤 다음처럼 접속한다. 파일명은 실제 지급 파일명으로 바꾼다.

```
ssh -i .\<사무국_발급키.pem> ubuntu@j15e201.p.ssafy.io
```

Linux 또는 macOS 에서는 접속 전에 키 권한을 chmod 600 <사무국_발급키.pem>으로 제한한다. SSH 접속에 실패하면 키가 이 서버에 등록됐는지, 서버 접속 허용 IP 와 사용자명이 맞는지 서버 관리 담당자에게 확인한다. 접속이 되더라도 Docker 명령 권한이 없으면 담당자의 권한 부여가 필요하다.

접속 후 컨테이너 이름과 DB 명을 먼저 확인한다. 현재 실행 중인 백엔드가 app_db 를 사용하고, PostgreSQL 컨테이너 이름은 local-route-personalization-postgres-1 이었다. 운영 구성이 바뀌면 아래 이름을 실제 값으로 바꾼다.

```
docker ps --format '{{.Names}} {{.Image}}'
umask 077
dump_file="$HOME/app_db_$(date -u +%Y%m%dT%H%M%S).dump"
docker exec local-route-personalization-postgres-1 \
  pg_dump -U postgres -Fc app_db > "$dump_file"
docker exec -i local-route-personalization-postgres-1 pg_restore -1 < "$dump_file"
sha256sum "$dump_file"
```

셸에 표시된 \$dump_file 경로를 확인한다. 로컬 PC 에는 scp -i .\<사무국_발급키.pem>

ubuntu@j15e201.p.ssafy.io:<서버_덤프_경로> .\exec\로 복사한다. Windows 에서는 Get-

FileHash .\exec\<덤프파일> -Algorithm SHA256 으로 서버 해시와 비교한다. 원본 전송은 제출처가 정한 보호된 경로로만 진행한다.

복원 담당자 확인 사항

1. 수신 파일 SHA-256 이 위 값과 같은지 비교한다.
2. PostgreSQL 16 환경에서 `pg_restore -1 <덤프파일>`로 항목 목록을 읽는다.
3. 격리된 새 DB 에 복원해 테이블과 필요한 데이터가 열리는지 확인한다. 운영 DB 에 바로 덮어쓰지 않는다.
4. 사용이 끝난 사본의 보관 기간과 삭제 책임자를 정한다.

기존 `infra/personalization/scripts/backup-postgres.sh` 는 `app_db`, `airflow_db`, `mlflow_db` 와 별도 설문 DB 를 주기적으로 백업한다. 이번 파일은 이 요청을 위해 별도로 만든 최신 스냅샷이다.

`backend/src/main/resources/db/migration/`의 Flyway SQL 은 스키마 변경 이력이며 현재 데이터 덤프를 대체하지 않는다.

4. 여행 만들기 시연 시나리오

시연 방식은 **앱에서 여행 만들기 1 회**다. 로그인된 시연 계정과 네트워크를 준비하고, 시연 날짜에 실제로 선택 가능한 여행 날짜·출발지·장소를 미리 확인한다. 화면과 서버 응답은 시연 환경에서 아직 직접 확인하지 않았다.

순서	앱 화면과 조작	발표할 내용·확인할 결과
1	앱 실행 후 로그인. 홈의 여행 계획 시작 영역에서 출발지, 날짜, 인원 을 정하고 일정 물어보기 로 진행한다.	한 번 입력한 기본 정보가 다음 여행 조건 화면으로 이어진다.
2	여행 조건 화면에서 여행 범위, 총예산, 이동 수단, 여행 카테고리, 여행 기분, 이번 여행 이동 보조·짐, 꼭 가고 싶은 장소 를 순서대로 선택한다. 화면에 추가 질문이 나오면 시연 선택값에 맞게 답하거나 건너뛴다.	선택한 조건을 요약 화면에서 다시 확인하고 필요하면 각 항목의 수정 을 누른다.
3	이 조건으로 일정 만들기 를 누른다. 생성 진행 화면에서 단계와 진행률을 보여 준다.	생성 요청은 매번 새 작업 ID 를 받는다. 아래 웹 예시 URL 의 <code>jobId</code> 를 재사용하지 않는다.
4	완료된 승차권을 눌러 뒤집고 뒷면의 일정 보기 를 누른다. 추천 코스가 나오면 하나를 고른 뒤 해당 코스 일정 보기 로 이동한다.	생성된 여행의 날짜별 장소와 동선, 예상 비용을 보여 준다. 추천 결과가 1 개라면 그 1 개를 연다.
5	일정 화면에서 순서 수정 을 누르고 장소 순서를 바꾼 뒤 저장 한다. 필요하면 장소 제외 등 화면에 제공되는 편집을 1 회 보여 준다.	변경 후 일정이 반영됐는지 확인한다. 앱 소스에는 별도의 「내 여행으로 확정」 버튼이 없다 . 이미 저장된 일정이므로 확정 버튼을 누른다고 설명하지 않는다.
6	하단의 내 여행 으로 이동해 방금 만든 여행을 찾고 연다.	여기서 여행 만들기 시연을 마치고 내 여행 기능 발표로 넘긴다.

웹 화면 대응 경로

- 홈·로그인·여행 만들기 진입: <https://j15e201.p.ssafy.io/>
- 기본 정보와 여행 조건: <https://j15e201.p.ssafy.io/plan>
- 여행 생성 진행: `/plan/generating?jobId=<이번 요청에서 새로 받은 ID>`
- 추천 코스와 일정: 앱 내부의 `/trips/<ID>/recommendations`, `/trips/<일정 ID>/itinerary` 화면

위 웹 주소는 화면 대응을 위한 참고 경로다. 제출자가 제공한 예시 `jobId=f1082660-119d-4606-9f38-53c8daad6386` 은 과거 요청 식별자로 보이므로 시연 링크로 고정하지 않는다.

진행이 지연되거나 실패할 때

- 생성 중 시간이 길어지면 진행 화면의 상태를 설명한다. 앱의 **백그라운드에서 계속**을 사용할 수 있고 완성된 여행은 **내 여행**에서 확인한다.
- 생성 실패 안내가 뜨면 **조건 다시 확인하기**로 돌아가 입력과 네트워크를 점검한다. 성공한 것처럼 다음 단계로 넘기지 않는다.
- 생성은 1 회만 수행한다. 발표 중 재시도를 실제로 했다면 시연 횟수와 이유를 명확히 말한다.

근거 화면: `frontend/app/(tabs)/home.tsx`, `frontend/app/(plan)/questions.tsx`,
`frontend/app/(plan)/generating.tsx`, `frontend/app/trips/[id]/recommendations.tsx`,
`frontend/app/trips/[id]/itinerary.tsx`, `frontend/app/(tabs)/trips.tsx`.